Object Storage: Buckets and Objects

Lab submission, Big Data Analytics

Name	Rayyan Jafar
Roll No	BSDSF23M031
Subject	Big Data Analytics
Date	19 October 2026
Tools	Python 3.13.5, boto3 1.43, moto 5.2.4 (moto_server on 127.0.0.1:5000), VS Code + Jupyter

All code below was run in order in one Jupyter session against a local S3-compatible server (moto). Each question shows the code as text, the printed output, and a one-line answer. Screenshots of the output are included for Questions 2, 4, 7, 9, 11, 12, 13, 14 and 15, as required.

SETUP: CONNECT TO THE LOCAL SERVER

```
import os, hashlib
from datetime import datetime, timedelta, timezone
import boto3
from botocore.exceptions import ClientError

s3 = boto3.client(
    "s3",
    endpoint_url="http://127.0.0.1:5000",
    aws_access_key_id="test",
    aws_secret_access_key="test",
    region_name="us-east-1",
)
```

Part A: Create

Question 1

Create a bucket named **hospital-scans**, then list all buckets and print their names.

CODE

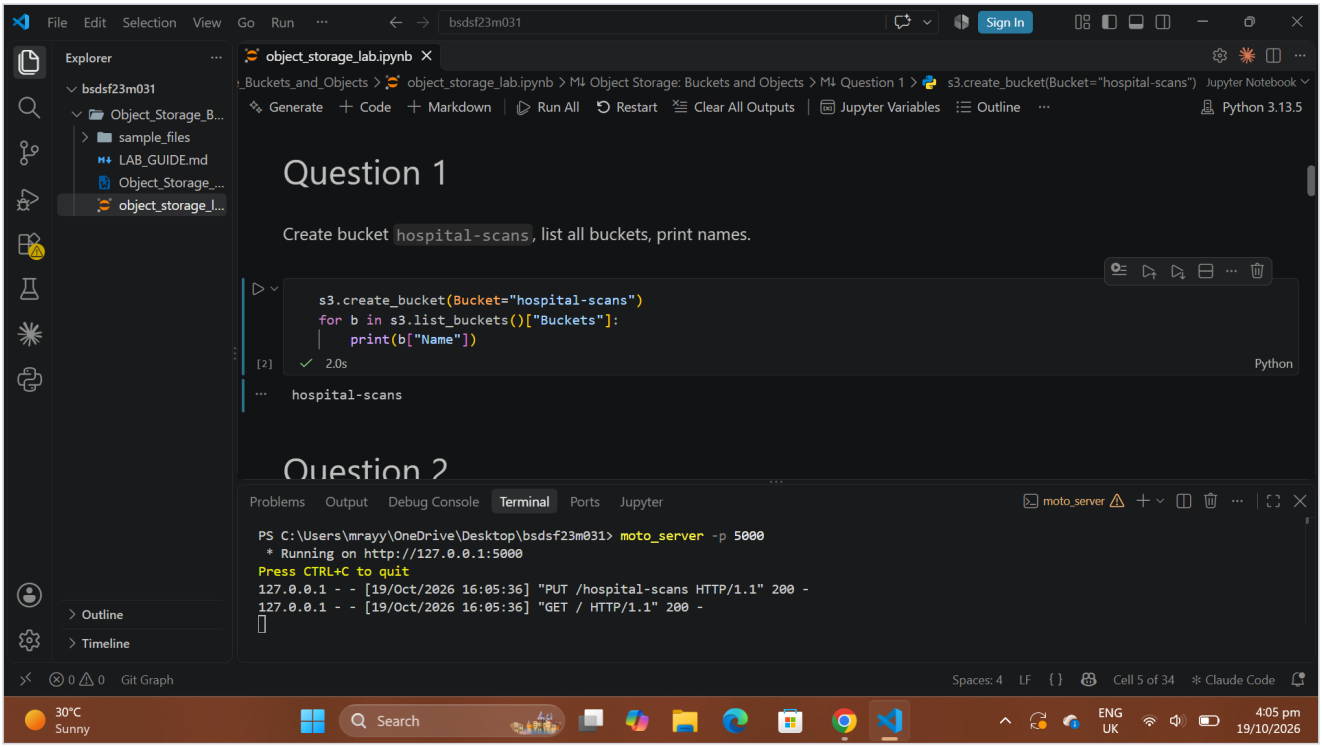
```
s3.create_bucket(Bucket="hospital-scans")
for b in s3.list_buckets()["Buckets"]:
    print(b["Name"])
```

OUTPUT

```
hospital-scans
```

Answer: Bucket created = **hospital-scans**

SCREENSHOT OF OUTPUT



Question 2

Upload **sample_files/xray-chest.png** with the key **patients/P-40231/2026-09-01/xray-chest.png**, then list the objects and print each key with its size in bytes.

CODE

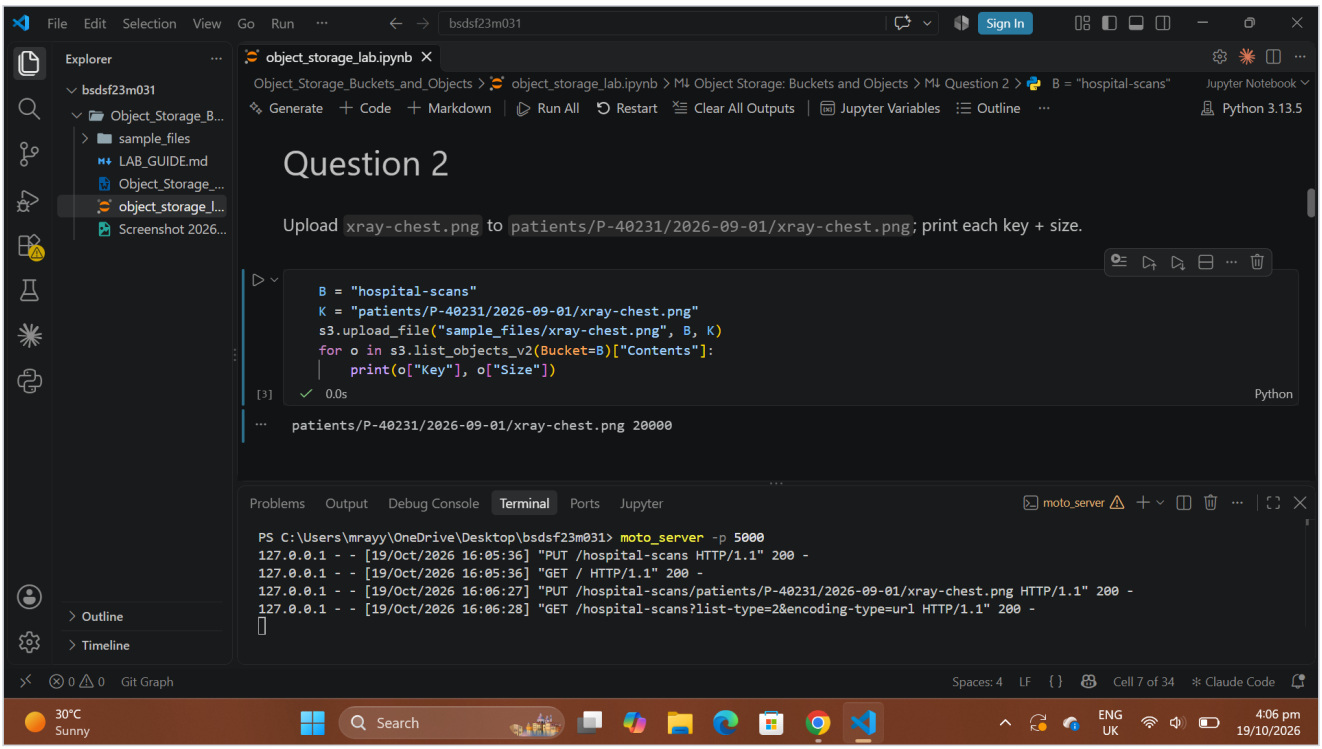
```
B = "hospital-scans"
K = "patients/P-40231/2026-09-01/xray-chest.png"
s3.upload_file("sample_files/xray-chest.png", B, K)
for o in s3.list_objects_v2(Bucket=B)["Contents"]:
    print(o["Key"], o["Size"])
```

OUTPUT

```
patients/P-40231/2026-09-01/xray-chest.png 20000
```

Answer: Uploaded object = **patients/P-40231/2026-09-01/xray-chest.png**, size = **20000 bytes**

SCREENSHOT OF OUTPUT



Question 3

Upload the other three scans with their content type and the metadata entries **patient** and **scan-type**, then print how many objects the bucket holds.

CODE

```
uploads = [
    ("lab-report.pdf", "patients/P-40231/2026-09-01/lab-report.pdf", "application/pdf",   "P-40231", "lab"),
    ("mri-brain.dcm",  "patients/P-40231/2026-09-15/mri-brain.dcm",  "application/dicom", "P-40231", "mri"),
    ("xray-wrist.png", "patients/P-40232/2026-09-03/xray-wrist.png", "image/png",         "P-40232", "xray"),
]
for f, key, ctype, patient, scan in uploads:
    s3.upload_file(f"sample_files/{f}", B, key,
                   ExtraArgs={"ContentType": ctype,
                              "Metadata": {"patient": patient, "scan-type": scan}})
print(s3.list_objects_v2(Bucket=B)["KeyCount"])
```

OUTPUT

```
4
```

Answer: Objects in bucket = **4**

Part B: Read

Question 4

List all objects in hospital-scans. Print the number of objects, then each key.

CODE

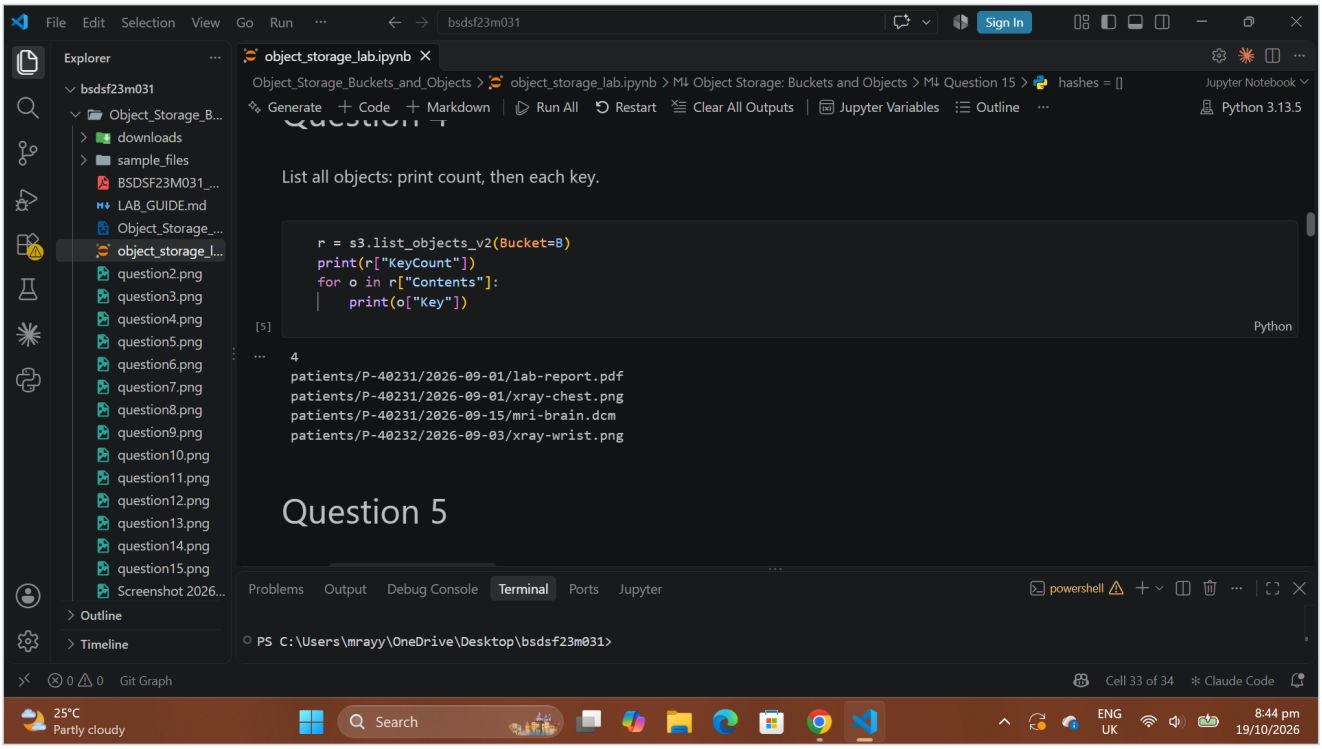
```
r = s3.list_objects_v2(Bucket=B)
print(r["KeyCount"])
for o in r["Contents"]:
    print(o["Key"])
```

OUTPUT

```
4
patients/P-40231/2026-09-01/lab-report.pdf
patients/P-40231/2026-09-01/xray-chest.png
patients/P-40231/2026-09-15/mri-brain.dcm
patients/P-40232/2026-09-03/xray-wrist.png
```

Answer: Objects in bucket = **4** (keys are returned in alphabetical order)

SCREENSHOT OF OUTPUT



Question 5

List only patient P-40231's objects using a prefix. Print the count, then each key with its size.

CODE

```
r = s3.list_objects_v2(Bucket=B, Prefix="patients/P-40231/")
print(r["KeyCount"])
for o in r["Contents"]:
    print(o["Key"], o["Size"])
```

OUTPUT

```
3
patients/P-40231/2026-09-01/lab-report.pdf 6000
patients/P-40231/2026-09-01/xray-chest.png 20000
patients/P-40231/2026-09-15/mri-brain.dcm 50000
```

Answer: Objects for patient P-40231 = **3** (lab-report 6000 B, xray-chest 20000 B, mri-brain 50000 B)

Question 6

List the top-level "folders" directly under **patients/** using a delimiter.

CODE

```
r = s3.list_objects_v2(Bucket=B, Prefix="patients/", Delimiter="/")
for c in r["CommonPrefixes"]:
    print(c["Prefix"])
```

OUTPUT

```
patients/P-40231/
patients/P-40232/
```

Answer: Prefixes under patients/ = **patients/P-40231/** and **patients/P-40232/** (key groups, not real folders)

Question 7

Without downloading it, print the size, content type and metadata of the MRI scan.

CODE

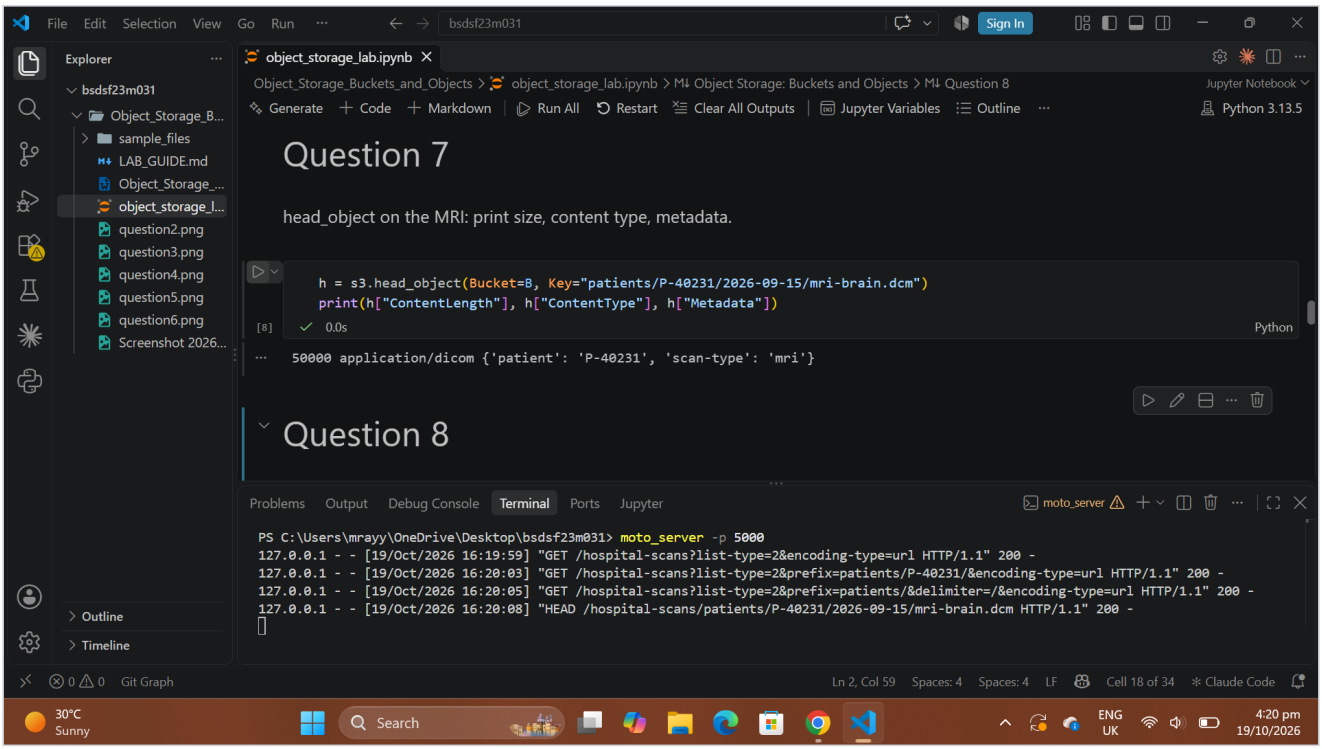
```
h = s3.head_object(Bucket=B, Key="patients/P-40231/2026-09-15/mri-brain.dcm")
print(h["ContentLength"], h["ContentType"], h["Metadata"])
```

OUTPUT

```
50000 application/dicom {'patient': 'P-40231', 'scan-type': 'mri'}
```

Answer: MRI = **50000 bytes**, **application/dicom**, metadata **patient=P-40231, scan-type=mri** (read with head_object, nothing downloaded)

SCREENSHOT OF OUTPUT



Question 8

Download the chest X-ray into **downloads/** and check that its MD5 hash matches the original.

CODE

```
os.makedirs("downloads", exist_ok=True)
s3.download_file(B, K, "downloads/xray-chest.png")

def md5(path):
    with open(path, "rb") as f:
        return hashlib.md5(f.read()).hexdigest()

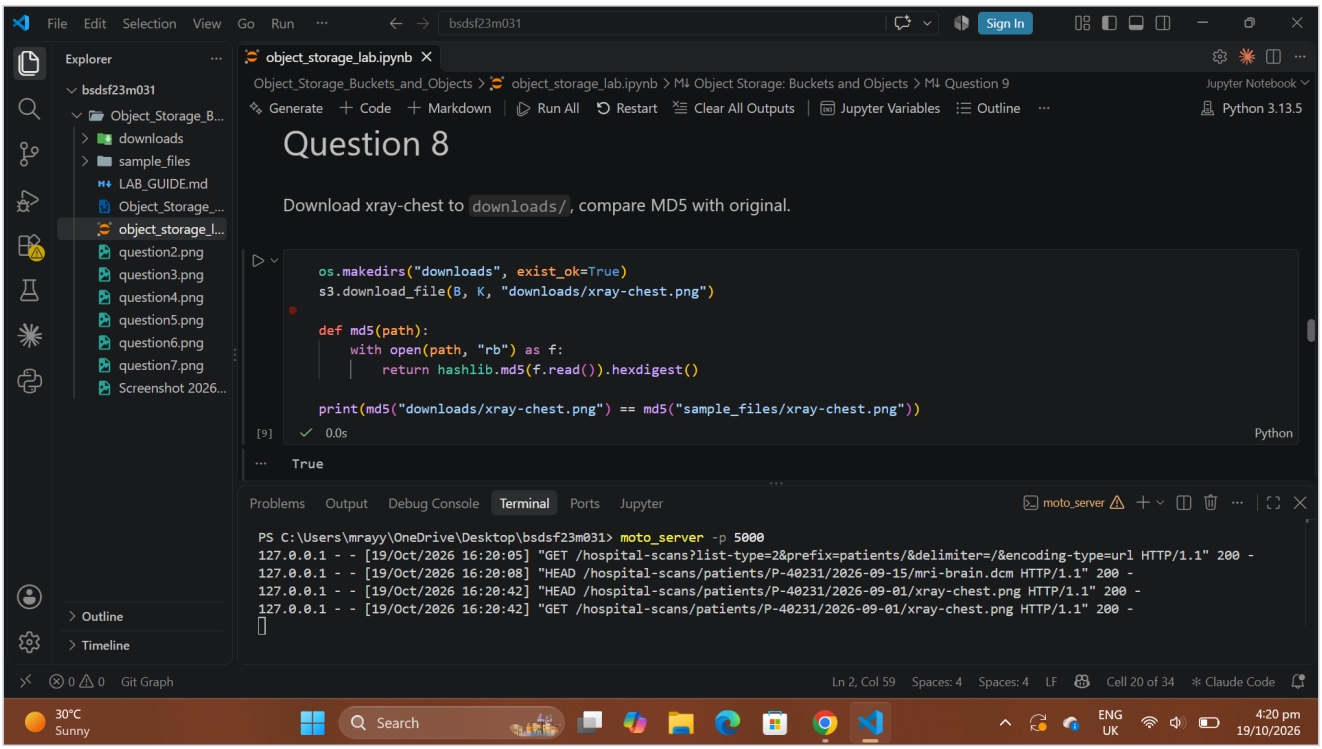
print(md5("downloads/xray-chest.png") == md5("sample_files/xray-chest.png"))
```

OUTPUT

```
True
```

Answer: MD5 hashes match = **True**, so the downloaded file is identical to the original

SCREENSHOT OF OUTPUT



Part C: Change an object

Question 9

Overwrite the Question 2 object with **xray-chest-v2.png**. Print the size before and after, and whether the ETag changed.

CODE

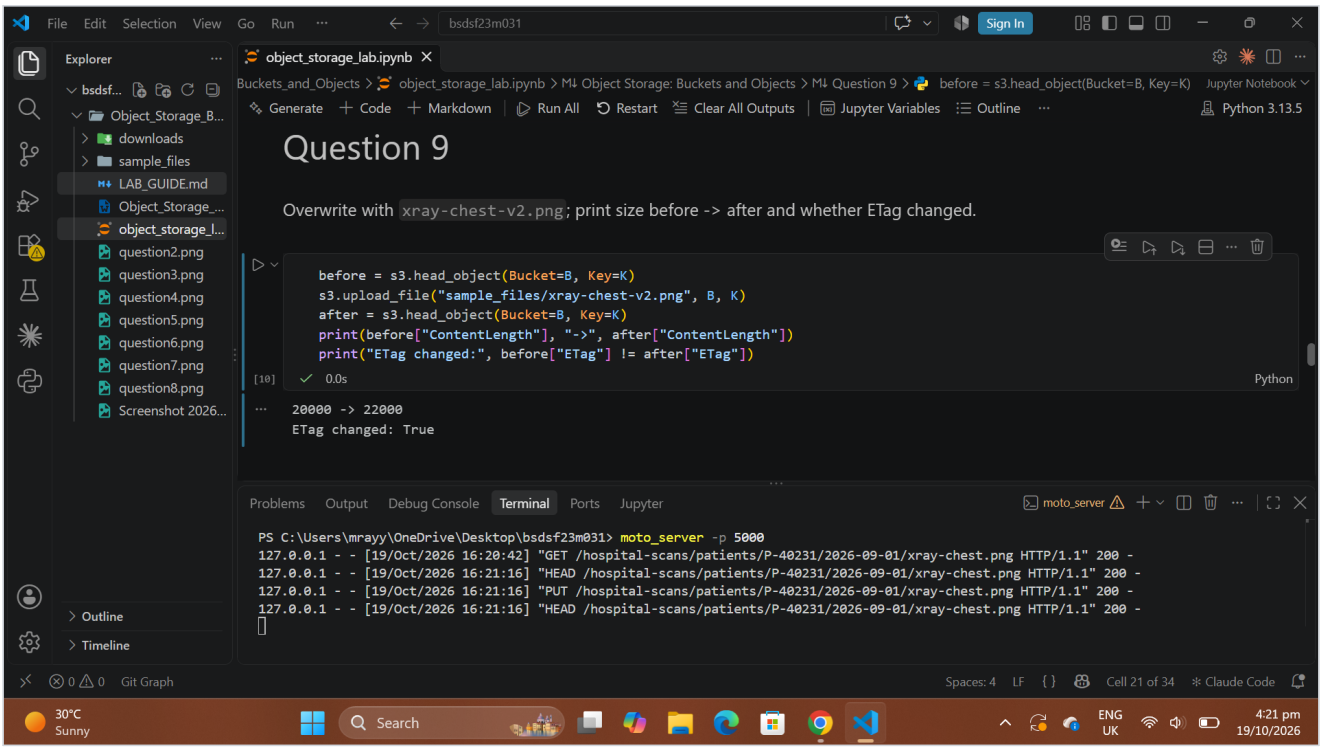
```
before = s3.head_object(Bucket=B, Key=K)
s3.upload_file("sample_files/xray-chest-v2.png", B, K)
after = s3.head_object(Bucket=B, Key=K)
print(before["ContentLength"], "->", after["ContentLength"])
print("ETag changed:", before["ETag"] != after["ETag"])
```

OUTPUT

```
20000 -> 22000
ETag changed: True
```

Answer: Size **20000 -> 22000** bytes, ETag changed = **True** (old content is gone, since versioning is off)

SCREENSHOT OF OUTPUT



Question 10

Move the lab report to **archive/P-40231/lab-report.pdf** by copying it and deleting the original. Print all keys afterwards.

CODE

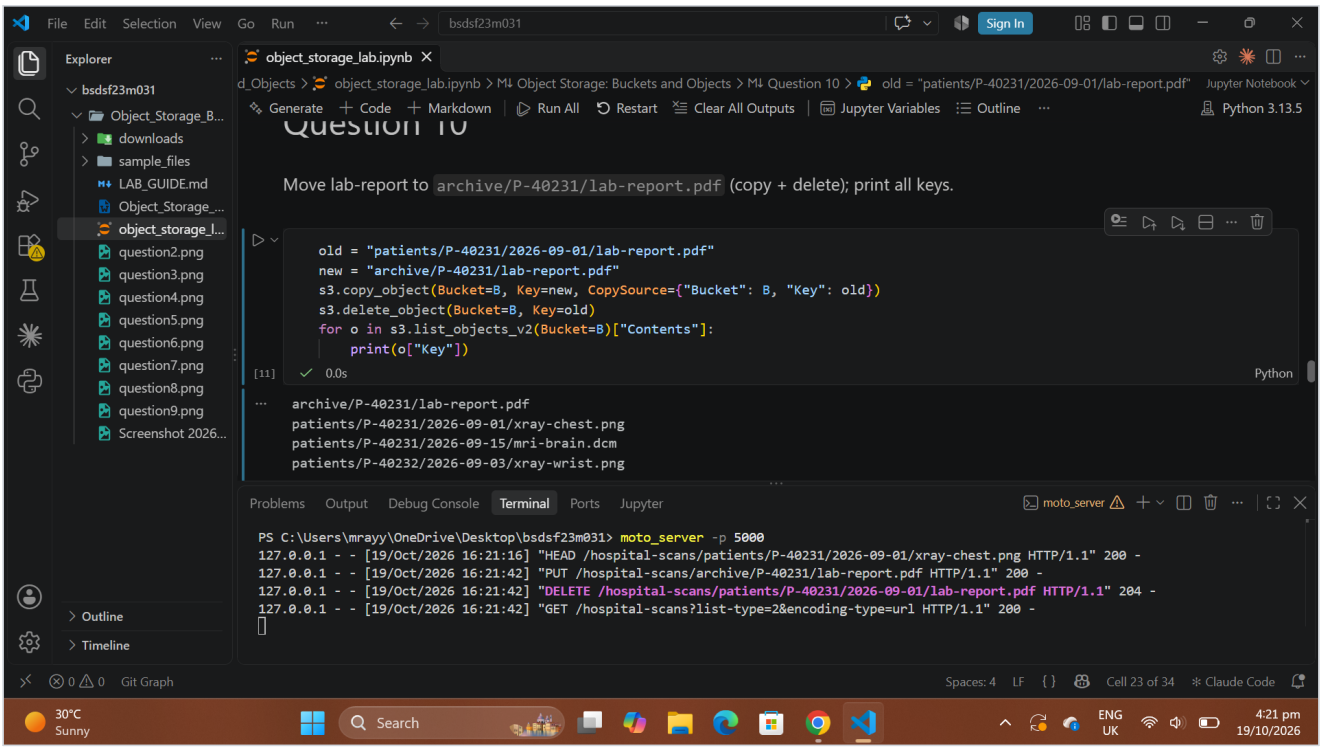
```
old = "patients/P-40231/2026-09-01/lab-report.pdf"
new = "archive/P-40231/lab-report.pdf"
s3.copy_object(Bucket=B, Key=new, CopySource={"Bucket": B, "Key": old})
s3.delete_object(Bucket=B, Key=old)
for o in s3.list_objects_v2(Bucket=B)["Contents"]:
    print(o["Key"])
```

OUTPUT

```
archive/P-40231/lab-report.pdf
patients/P-40231/2026-09-01/xray-chest.png
patients/P-40231/2026-09-15/mri-brain.dcm
patients/P-40232/2026-09-03/xray-wrist.png
```

Answer: Lab report now at **archive/P-40231/lab-report.pdf**, and the bucket still holds **4** keys

SCREENSHOT OF OUTPUT



Question 11

Turn on versioning, upload the original xray-chest.png again to the same key, and print each version's id, whether it is the latest, and its size.

CODE

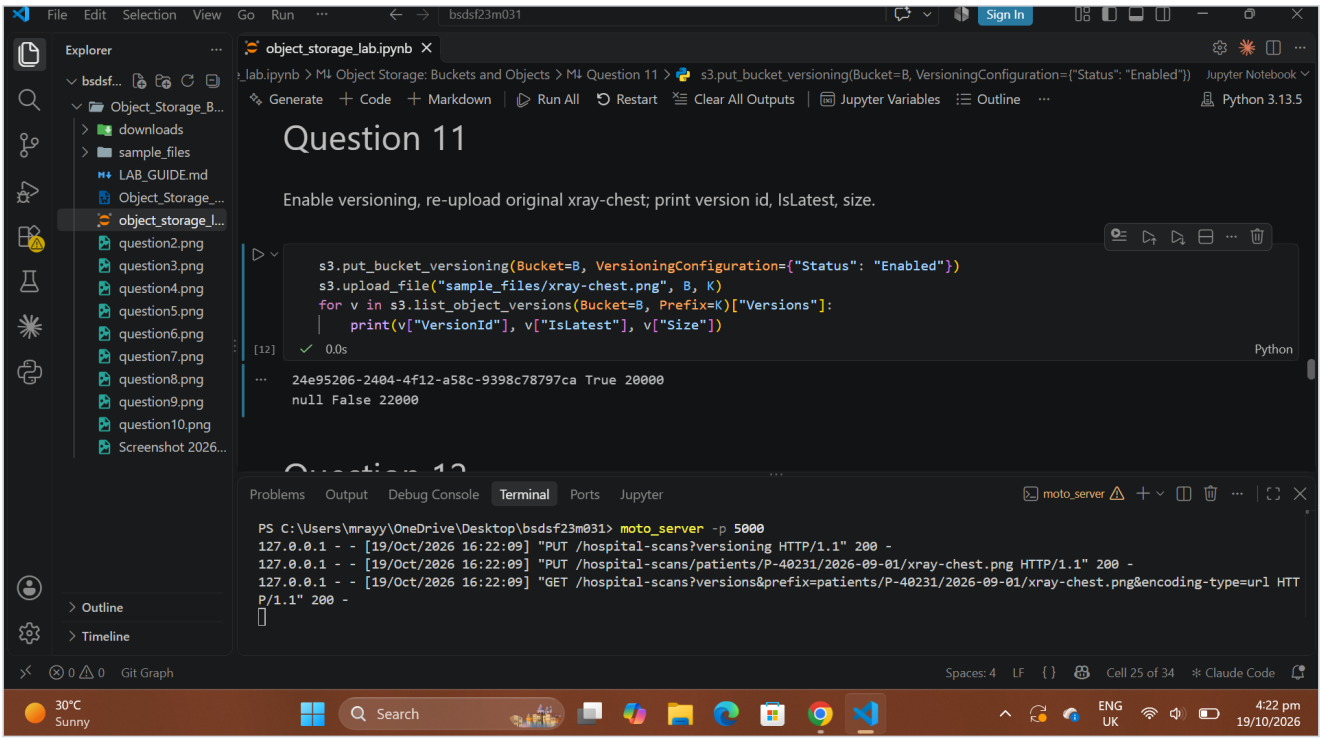
```
s3.put_bucket_versioning(Bucket=B, VersioningConfiguration={"Status": "Enabled"})
s3.upload_file("sample_files/xray-chest.png", B, K)
for v in s3.list_object_versions(Bucket=B, Prefix=K)["Versions"]:
    print(v["VersionId"], v["IsLatest"], v["Size"])
```

OUTPUT

```
24e95206-2404-4f12-a58c-9398c78797ca True 20000
null False 22000
```

Answer: 2 versions: the new upload (latest, 20000 B) and the pre-versioning copy (id **null**, 22000 B)

SCREENSHOT OF OUTPUT



Part D: Delete and protect

Question 12

For **xray-wrist.png**: (a) delete it and check for a delete marker, (b) count visible objects, (c) count versions and delete markers, (d) remove the delete marker and count again.

CODE

```
W = "patients/P-40232/2026-09-03/xray-wrist.png"

# (a)
r = s3.delete_object(Bucket=B, Key=W)
print("(a)", r.get("DeleteMarker", False))

# (b)
print("(b)", s3.list_objects_v2(Bucket=B)["KeyCount"])

# (c)
r = s3.list_object_versions(Bucket=B, Prefix=W)
versions, markers = r.get("Versions", []), r.get("DeleteMarkers", [])
print("(c)", len(versions), "version and", len(markers), "delete marker")

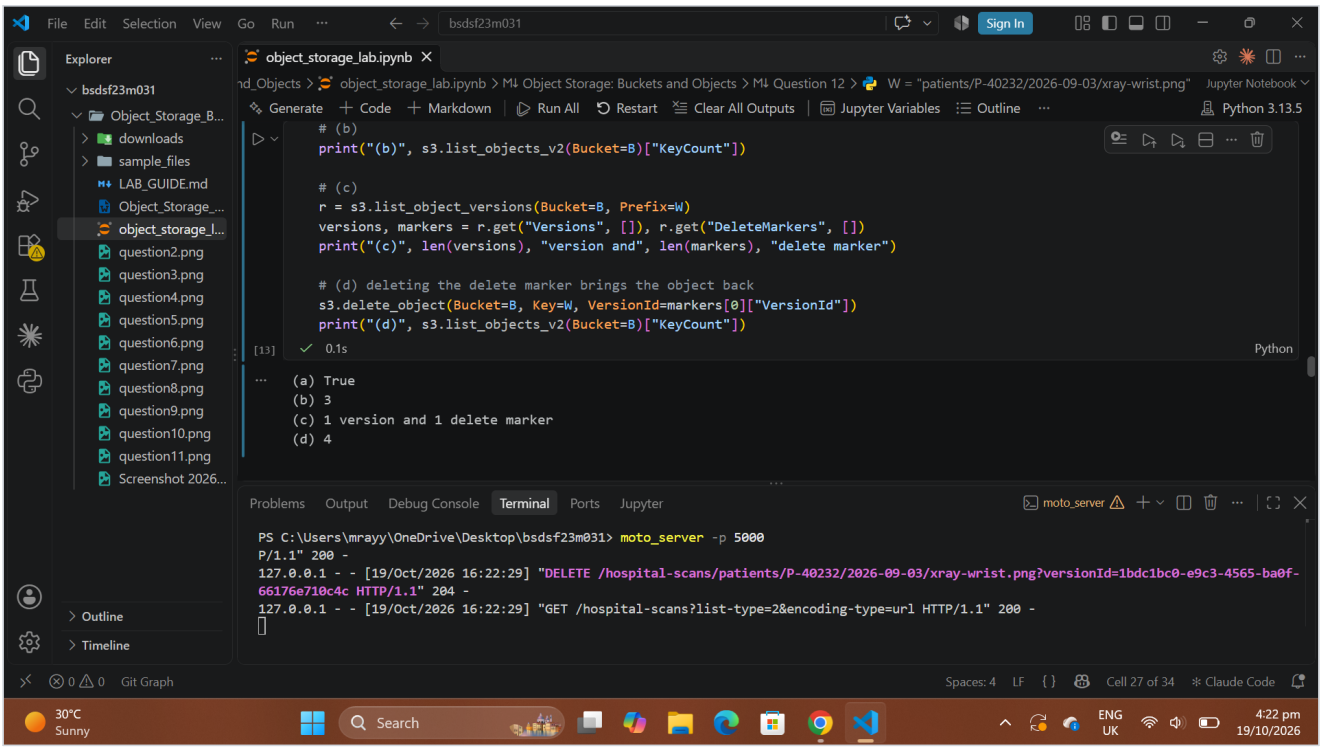
# (d) deleting the delete marker brings the object back
s3.delete_object(Bucket=B, Key=W, VersionId=markers[0]["VersionId"])
print("(d)", s3.list_objects_v2(Bucket=B)["KeyCount"])
```

OUTPUT

```
(a) True
(b) 3
(c) 1 version and 1 delete marker
(d) 4
```

Answer: Delete marker = **True**, visible objects = **3**, **1 version + 1 delete marker**, restored count = **4**

SCREENSHOT OF OUTPUT



Question 13

Create **bank-statements** with object lock enabled. (a) Upload the statement locked in COMPLIANCE mode for 30 days, (b) print the lock mode and date, (c) try to delete that version, (d) print the object count.

CODE

```
BB = "bank-statements"
SK = "statements/acct-771/2026-03.pdf"
s3.create_bucket(Bucket=BB, ObjectLockEnabledForBucket=True)

# (a)
until = datetime.now(timezone.utc) + timedelta(days=30)
with open("sample_files/statement-2026-03.pdf", "rb") as f:
    r = s3.put_object(Bucket=BB, Key=SK, Body=f,
                      ObjectLockMode="COMPLIANCE",
                      ObjectLockRetainUntilDate=until)
vid = r["VersionId"]

# (b)
h = s3.head_object(Bucket=BB, Key=SK)
print("(b)", h["ObjectLockMode"], h["ObjectLockRetainUntilDate"])

# (c)
try:
    s3.delete_object(Bucket=BB, Key=SK, VersionId=vid)
    print("(c) deleted (lock did not work)")
except ClientError as e:
    print("(c)", e.response["Error"]["Code"])

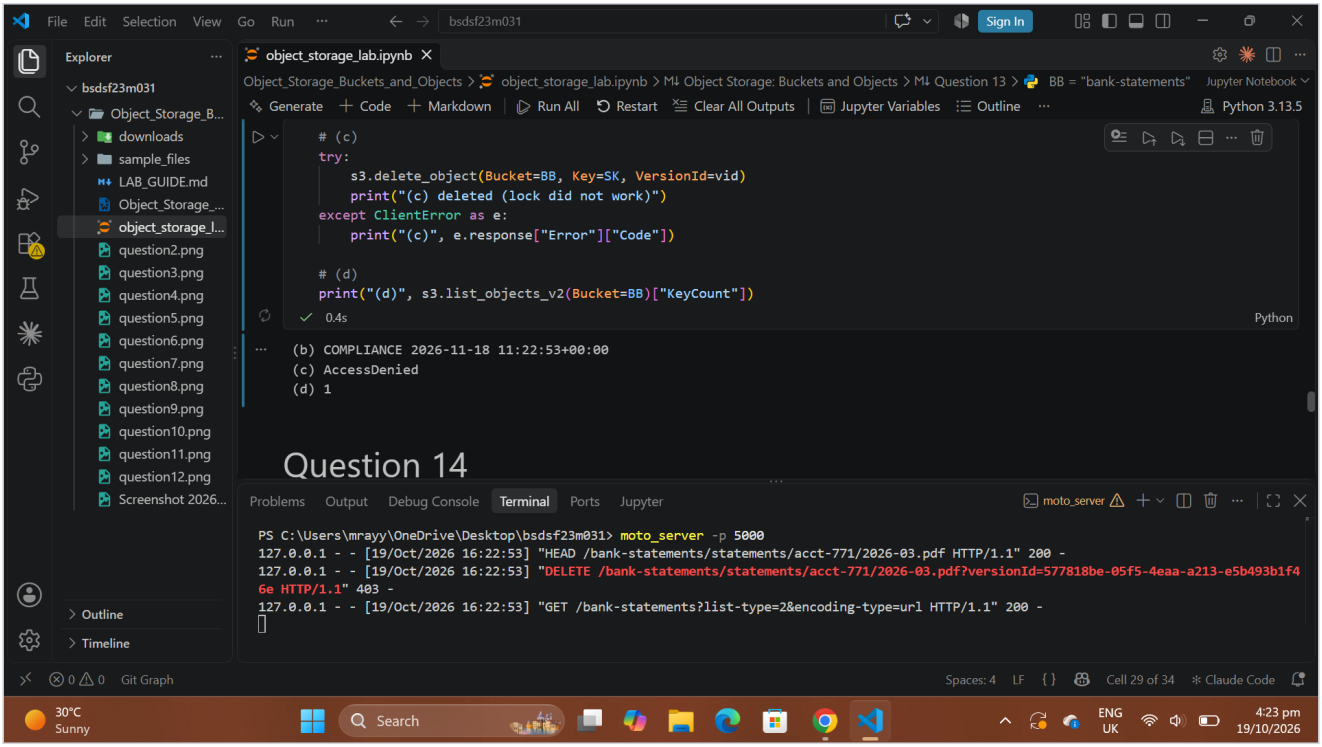
# (d)
print("(d)", s3.list_objects_v2(Bucket=BB)["KeyCount"])
```

OUTPUT

```
(b) COMPLIANCE 2026-11-18 11:22:53+00:00
(c) AccessDenied
(d) 1
```

Answer: Lock = **COMPLIANCE** until **2026-11-18**, delete attempt = **AccessDenied**, objects in bucket = **1**

SCREENSHOT OF OUTPUT



Part E: Photo sizes and chunking

Question 14

Create **photo-app** and upload three sizes of photo 8831. (a) List the keys and sizes, (b) write **get_photo(photo_id, screen)**, (c) compare the bytes needed to scroll 50 photos with feed copies versus originals.

CODE

```
PB = "photo-app"
s3.create_bucket(Bucket=PB)
for f, name in [("photo-original.jpg", "original"),
                ("photo-feed.jpg", "feed"),
                ("photo-thumb.jpg", "thumb")]:
    s3.upload_file(f"sample_files/{f}", PB, f"photos/8831/{name}.jpg",
                   ExtraArgs={"ContentType": "image/jpeg"})

# (a)
print("(a)")
for o in s3.list_objects_v2(Bucket=PB, Prefix="photos/8831/")["Contents"]:
    print(o["Key"], o["Size"])

# (b)
def get_photo(photo_id, screen):
    size = {"list": "thumb", "feed": "feed", "fullscreen": "original"}[screen]
    return s3.get_object(Bucket=PB, Key=f"photos/{photo_id}/{size}.jpg")["Body"].read()

print("(b)")
for screen in ["list", "feed", "fullscreen"]:
    print(screen, len(get_photo(8831, screen)))

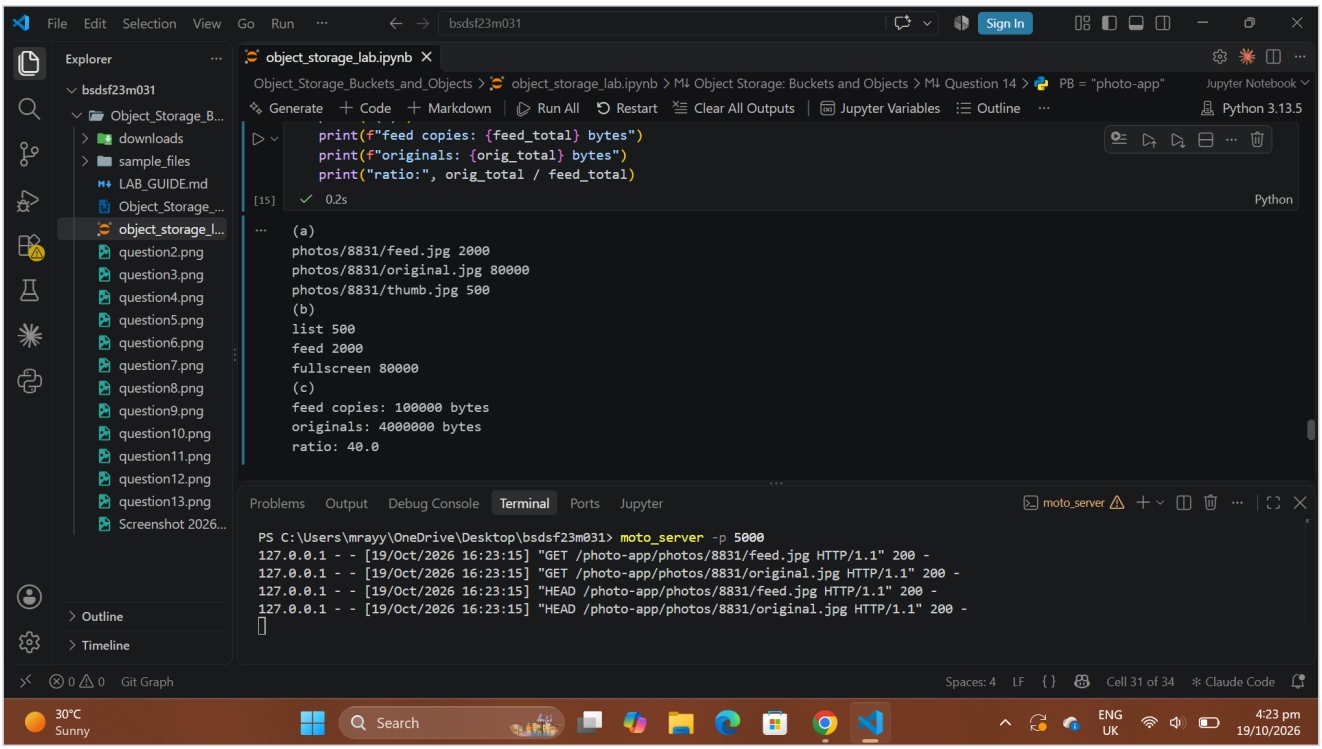
# (c) sizes from head_object, nothing downloaded
feed_total = s3.head_object(Bucket=PB, Key="photos/8831/feed.jpg")["ContentLength"] * 50
orig_total = s3.head_object(Bucket=PB, Key="photos/8831/original.jpg")["ContentLength"] * 50
print("(c)")
print(f"feed copies: {feed_total} bytes")
print(f"originals: {orig_total} bytes")
print("ratio:", orig_total / feed_total)
```

OUTPUT

```
(a)
photos/8831/feed.jpg 2000
photos/8831/original.jpg 80000
photos/8831/thumb.jpg 500
(b)
list 500
feed 2000
fullscreen 80000
(c)
feed copies: 100000 bytes
originals: 4000000 bytes
ratio: 40.0
```

Answer: 50-photo scroll = **100000 bytes** (feed) vs **4000000 bytes** (originals). Originals are **40x** larger

SCREENSHOT OF OUTPUT



Question 15(a)

(a) Split **textbook.bin** into 4096-byte chunks, hash each one with MD5, and print the total chunks, unique chunks and the storage saving.

CODE

```
hashes = []
with open("sample_files/textbook.bin", "rb") as f:
    while True:
        chunk = f.read(4096)
        if not chunk:
            break
        hashes.append(hashlib.md5(chunk).hexdigest())

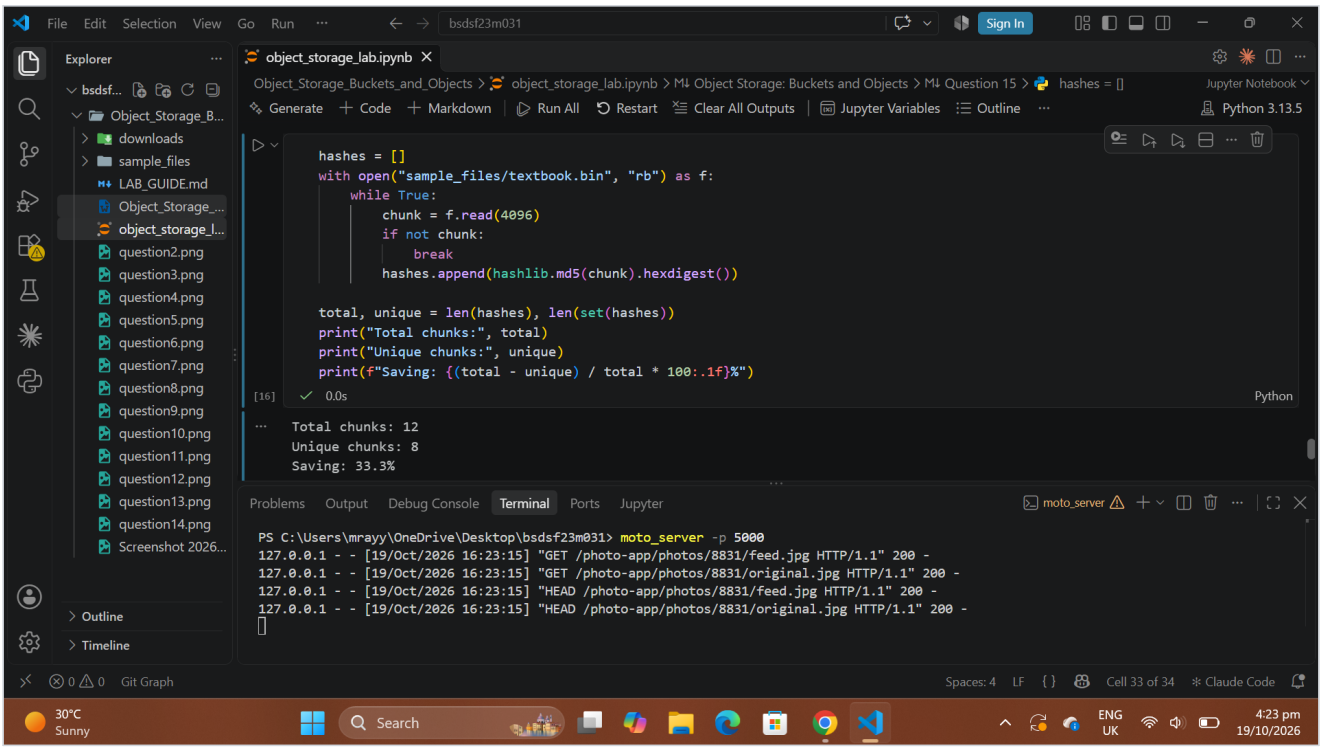
total, unique = len(hashes), len(set(hashes))
print("Total chunks:", total)
print("Unique chunks:", unique)
print(f"Saving: {(total - unique) / total * 100:.1f}%")
```

OUTPUT

```
Total chunks: 12
Unique chunks: 8
Saving: 33.3%
```

Answer: Total chunks = **12**, unique chunks = **8**, saving = **33.3%**

SCREENSHOT OF OUTPUT



Question 15(b)

Why does chunking save space when 1,000 students upload the same textbook?

Each chunk is identified by the MD5 hash of its bytes. Identical content always gives the same hash, so the drive can tell whether it already has a chunk by looking up its hash instead of comparing bytes.

When the first student uploads the textbook, the drive stores its 8 unique chunks plus a short list of the 12 hashes that rebuild the file. When the next 999 students upload the same file, every hash is already in the store. For each of them the drive saves only that list of hashes, which points to the existing chunks, and writes no new chunk bytes. The client can also send the hashes first and skip uploading bytes the server already has, which saves bandwidth as well as disk.

So instead of 1,000 full copies (1,000 x 49,152 bytes, about 49 MB at this sample's scale), the drive keeps roughly one copy's worth of chunks (8 x 4,096 = 32,768 bytes) plus 1,000 small hash lists. My result in part (a) shows the same idea inside a single file: 4 of the 12 chunks repeat, so storing each unique chunk once saves 33.3% even for one upload. Across many users uploading the same file the saving gets close to 99.9%.